Day 17 – Insertion Sort

🚀 Insertion Sort – Concept

💡 Basic Idea

👉 Array ko 2 parts me divide karo:

Sorted part | Unsorted part

👉 Har step me:

Unsorted se ek element uthao

Usko sorted part me correct position par insert karo

🧠 Real-Life Example

👉 Playing cards sort karte ho na?

- ✓ Ek card uthaya
- ✓ Usko sahi jagah par insert kiya

👉 Same process = Insertion Sort 🃏

🔁 Dry Run (Step-by-Step)

Example:

arr = [5, 3, 4, 1]

🔹 Pass 1 (i = 1)

key = 3

Compare with 5:

[3, 5, 4, 1]

🔹 Pass 2 (i = 2)

key = 4

Insert between 3 & 5:

[3, 4, 5, 1]

- Pass 3 (i = 3)

key = 1

Shift sab:

[1, 3, 4, 5]

✅ Final Output

1 3 4 5

💻 Java Code (Insertion Sort)

```
package Day_17;

public class InsertionSort {

    public static void insertionsort(int[] arr){
        int n=arr.length;
         for(int i =1; i<n; i++){
            int key = arr[i];
            int j =i-1;

            while (j>=0 && arr[j]>key) {
                arr[j+1] = arr[j];
                j--;
            }
            arr[j+1] =key;
         }

    }

    public static void main(String[] args) {
        int[] arr={5,3,4,1};

        insertionsort(arr);

        for (int nums  : arr) {
            System.out.println(nums +"");
        }
    }

}
```

🔄 Descending Order

👉 Bas condition change:

while(j >= 0 && arr[j] < key)

```java
package Day_17;

public class InsertionSortDec {

    public static void insertionsortdec(int[] arr){
        int n = arr.length;

        for(int i=1; i<n; i++){
            int key = arr[i];
            int j=i-1;

            while (j>=0 && arr[j]<key) {  // arr[j]<key hi decreasing order dega
                arr[j+1] = arr[j];
                j--;
            }
            arr[j+1] = key;
        }
    }

    public static void main(String[] args) {
        int[] arr ={2,4,5,3,7};

        insertionsortdec(arr);

        for (int i : arr) {
            System.out.println(i +"");
        }
    }

}
```

🧠 Important Points

✅ Stable sorting algorithm
✅ Adaptive (already sorted ho to fast)
✅ Small data ke liye best

⏱️ Time Complexity

Case	Time
Best (sorted)	O(n) ✅
Average	$O(n^2)$
Worst	$O(n^2)$

⚔️ Comparison (All 3)

Feature	Bubble	Selection	Insertion
Best Case	O(n)	$O(n^2)$	O(n)
Swap	zyada	kam	shift
Stable	Yes	No	Yes
Practical	No	No	Yes ✅

Problem :

Count shifts

```
package Day_17;

public class SwapCountInsertion {

    public static void swapCountIns(int[] arr){
        int n= arr.length;
        int totalSwap =0;

        for(int i=1; i<n; i++){
            int key =arr[i];
            int j=i-1;

            while (j>=0 && arr[j]<key) {
                arr[j+1] =arr[j];
                j--;
            }
            arr[j+1] = key;
            totalSwap++;
        }
        System.err.println("Total Count of Insertion Sort is :" +totalSwap);
    }

    public static void main(String[] args){
        int[] arr ={5,4,3,8,2,6,1};

        swapCountIns(arr);

        for (int i : arr) {
            System.out.println(i +"");
        }
    }

}
```